

DiceKriging vs RobustGaSP: 37D example

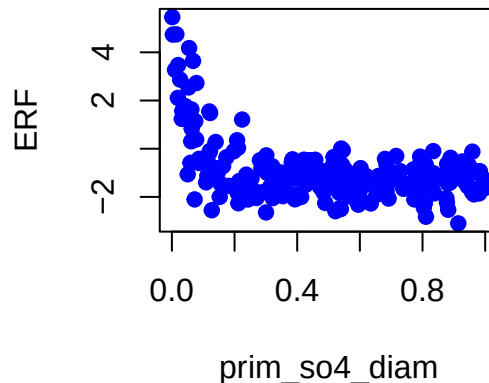
We repeat the verification steps in the 1D example but for data in 37 dimensions. We do it for two outputs, since we'll be wanting to find the alignment measure between a pair of outputs. The outputs are global ERF and AOD for a particular gridbox (first longitude -87.1875, latitude -28.125; next longitude 6.5625, latitude 34.375; and finally for longitude 100.3125, latitude -59.375), both for July, taken from the A-CURE monthly mean PPE from the first version of the U.K. Earth System Model (UKESM1).

We import the 221 (normalised) input settings
and create the matrix H .

Firstly for ERF

We import the outputs.

The 221 observations of ERF look like this when plotted against `prim_so4_diam`:



We fit an emulator using the `rgasp()` function in the `RobustGaSP` package:

```
rgasp(design = input,  
      response = output,  
      trend = H_matrix,  
      lower_bound=FALSE,  
      kernel_type = 'pow_exp', alpha = rep(2, dim_inputs),  
      zero.mean = "No"  
    )
```

Here, a constant “mean basis” has been used, with “no constraint on optimization” (as per the `RobustGaSP` package manual). A Gaussian (squared exponential) kernel has been used, and linear mean prior function. [Zero mean should still equal "No"?](#) We get the following output:

```
## The upper bounds of the range parameters are Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf
```

```
## The initial values of range parameters are 0.01879214 0.01878301 0.01879183 0.01878643 0.01879164 0.
## Start of the optimization 1 :
## The number of iterations is 2
## The value of the marginal posterior function is -731.8025
## Optimized range parameters are Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf
## Optimized nugget parameter is 0
## Convergence: TRUE
## The initial values of range parameters are 55.21773 54.79025 55.20323 54.94957 55.19415 54.96311 55.
## Start of the optimization 2 :
## The number of iterations is 205
## The value of the marginal posterior function is -423.0086
## Optimized range parameters are 11942837 131.0951 12047420 64.40433 34.41683 85.67004 1.535288 412.4
## Optimized nugget parameter is 0
## Convergence: TRUE
```

This can be contrasted with the Matern 5/2 kernel if required.

```
## The upper bounds of the range parameters are Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf
## The initial values of range parameters are 0.01879214 0.01878301 0.01879183 0.01878643 0.01879164 0.
## Start of the optimization 1 :
## The number of iterations is 2
## The value of the marginal posterior function is -731.8025
## Optimized range parameters are Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf
## Optimized nugget parameter is 0
## Convergence: TRUE
## The initial values of range parameters are 55.21773 54.79025 55.20323 54.94957 55.19415 54.96311 55.
## Start of the optimization 2 :
## The number of iterations is 128
## The value of the marginal posterior function is -399.1334
## Optimized range parameters are 171.2034 184.2074 3046466856 68.39438 19.64131 42.40712 2.309228 107
## Optimized nugget parameter is 0
## Convergence: TRUE
```

The posterior mean function for both kernels, as well as that obtained using both kernels in `DiceKriging`'s `km()` function, again with a linear mean prior function, are shown in Figure 1.

I should put in some validation (see 1D example).

The widths of the 95% credible intervals for the two packages are compared:

Package	Kernel	Mean width of 95% CI
RobustGaSP	Gauss	0.7207
RobustGaSP	Matern 5/2	0.6782
DiceKriging	Gauss	1.8097
DiceKriging	Matern 5/2	3.616

This idea was taken from the ‘RobustGaSP’ manual. Is a mean width of 1.81, or even 0.72, a problem, when the range of the output is 8.5569507?

Verifying the estimates by-hand for DiceKriging

We already have matrix H (defined earlier for use in the `rgasp` function's `trend` argument). We also need matrix A and its inverse, built using the estimate of the range parameter.

Finally, we need $\mathbf{t}(x^*)$ for all 1000 of the new inputs x^* .

We're now ready to calculate posterior predictions for each of the new input settings, using the posterior estimates from the two packages. For both we calculate the following:

$$\frac{\sum_{i=1}^{1000} |(\text{predictions from package})_i - (\text{predictions by-hand})_i|}{10^{-8} \times 1000}$$

and get, for `DiceKriging` and `RobustGaSP` respectively:

```
## [1] 1.165887
```

```
## [1] 4848.548
```

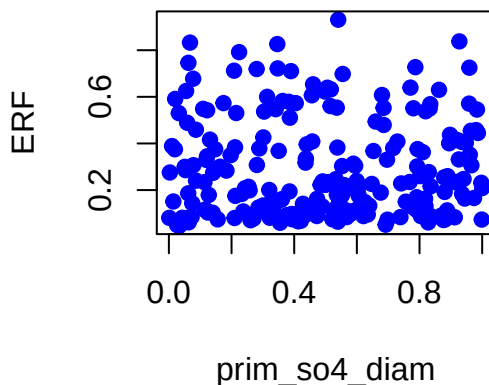
Are these OK? 0 – 2 with DK and in the 4000's with RG.

Finally, we create an object containing the normalised partial derivatives for use in the alignment measure.

Secondly for AOD

We import the outputs.

The 221 observations of ERF look like this when plotted against `prim_so4_diam`:



We fit an emulator using the `rgasp()` function in the `RobustGaSP` package:

```
rgasp(design = input,
      response = output,
      trend = H_matrix,
      lower_bound=FALSE,
      kernel_type = 'pow_exp', alpha = rep(2,dim_inputs),
      zero.mean = "No"
    )
```

Here, a constant “mean basis” has been used, with “no constraint on optimization” (as per the `RobustGaSP` package manual). A Gaussian (squared exponential) kernel has been used, and linear mean prior function. Zero mean should still equal "No"? We get the following output:

```
## The upper bounds of the range parameters are Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf
## The initial values of range parameters are 0.01879214 0.01878301 0.01879183 0.01878643 0.01879164 0.
## Start of the optimization 1 :
## The number of iterations is 2
## The value of the marginal posterior function is -265.6773
## Optimized range parameters are Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf
## Optimized nugget parameter is 0
## Convergence: TRUE
## The initial values of range parameters are 55.21773 54.79025 55.20323 54.94957 55.19415 54.96311 55.
## Start of the optimization 2 :
## The number of iterations is 152
## The value of the marginal posterior function is 191.2929
## Optimized range parameters are 124.8791 748.8938 39303177 4857534383509 161.049 310407165 154.0823 0
## Optimized nugget parameter is 0
## Convergence: FALSE
```

This can be contrasted with the Matern 5/2 kernel if required.

```
## The upper bounds of the range parameters are Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf
## The initial values of range parameters are 0.01879214 0.01878301 0.01879183 0.01878643 0.01879164 0.
## Start of the optimization 1 :
## The number of iterations is 2
## The value of the marginal posterior function is -265.6773
## Optimized range parameters are Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf Inf
## Optimized nugget parameter is 0
## Convergence: TRUE
## The initial values of range parameters are 55.21773 54.79025 55.20323 54.94957 55.19415 54.96311 55.
## Start of the optimization 2 :
## The number of iterations is 90
## The value of the marginal posterior function is 209.793
## Optimized range parameters are 298.0774 122673.9 67238.97 380.4762 224.7517 3233.683 423.0112 27.55
## Optimized nugget parameter is 0
## Convergence: TRUE
```

The posterior mean function for both kernels, as well as that obtained using both kernels in `DiceKriging`'s `km()` function, again with a linear mean prior function, are shown in Figure 1.

I should put in some validation (see 1D example).

The widths of the 95% credible intervals for the two packages are compared:

Package	Kernel	Mean width of 95% CI
RobustGaSP	Gauss	0.0363
RobustGaSP	Matern 5/2	0.0343
DiceKriging	Gauss	0.2609
DiceKriging	Matern 5/2	0.1153

This idea was taken from the ‘RobustGaSP’ manual. Is a mean width of 0.26, or even 0.04, a problem, when the range of the output is 0.8847598?

Verifying the estimates by-hand for DiceKriging

We already have matrix H (defined earlier for use in the `rgasp` function's `trend` argument). We also need matrix A and its inverse, built using the estimate of the range parameter.

Only for the second gridbox, I received the error Error in solve.default(A_matrix_RG) - system is computationally singular- reciprocal condition number = 1.69941e-17 so added a tiny nugget to the diagonal of A, then it inverted just fine.

Finally, we need $\mathbf{t}(x^*)$ for all 1000 of the new inputs x^* .

We're now ready to calculate posterior predictions for each of the new input settings, using the posterior estimates from the two packages. For both we calculate the following:

$$\frac{\sum_{i=1}^{1000} |(\text{predictions from package})_i - (\text{predictions by-hand})_i|}{10^{-8} \times 1000}$$

and get, for DiceKriging and RobustGaSP respectively:

```
## [1] 0.000003069064
```

```
## [1] 1064.247
```

Are these OK? Close to 1 (or 0) with DK and ~ 900 , $> 90,000,000$ and ~ 1000 (for the three gridboxes respectively) with RG.

Finally, we create an object containing the normalised partial derivatives for use in the alignment measure.

AM calculation

Finally we calculate the alignment measure.

Attempt	AM for ERF and AOD in gb 1	AM for ERF and AOD in gb 2	AM for ERF and AOD in gb 3
1	0.2815566	0.2118547	0.2562991
2	0.2825316	0.2118477	0.2601877
3	0.2652177	0.2117447	0.2601681
4	0.2647665	0.211873	0.2197906
5	0.2653364	0.2118245	0.2834103
6	0.2821098	0.1850545	0.2906794
7	0.2653951	0.2040215	0.2541661
8	0.2648363	0.2118733	0.2600939
9	0.2653367	0.1116973	0.2562923
10	0.2701372	0.2118717	0.2599827
Robust	0.01612035	0.2331373	0.01426525